

SWE-Bench 5G: Benchmarking AI Coding Agents on Telecom Network Engineering Tasks

Jiao Chen*, Jianhua Tang*, Xiaotong Yang[†], and Zuohong Lv[‡]

*Shenzhen Smart City Technology Development Group Company, Ltd.

[†]Shien-Ming Wu School of Intelligent Engineering, South China University of Technology

[‡]China Unicom Group Co., Ltd.

Abstract—AI coding agents have achieved remarkable progress on general-purpose software benchmarks, yet their ability to handle specification-driven vertical domains remains unexplored. We propose SWE-Bench 5G¹, the first benchmark designed to investigate whether AI coding agents can resolve real-world bugs in 5G core network software. The benchmark collects task instances from three open-source 5G projects, packages each as a self-contained Docker environment with automated fail-to-pass tests, and provides a dual test strategy tailored to the complex runtime dependencies of telecom code. In addition, for instances whose original issues reference 3GPP specification clauses, we construct concise specification context documents, enabling controlled evaluation of whether domain knowledge improves agent performance. Experiments on four LLMs reveal that all models diagnose bugs at rates exceeding 91%, yet resolve rates remain between 10% and 30%, indicating that iterative code editing capability, not domain understanding, is the primary bottleneck. The specification injection experiment further shows that 3GPP excerpts improve resolve rates on specification-dependent bugs but provide no benefit on generic defensive checks.

I. INTRODUCTION

Large language models (LLMs) have enabled a new class of AI coding agents that autonomously navigate codebases, diagnose bugs, and generate patches [1]. Benchmarks such as SWE-Bench [1] and its extensions have driven rapid progress on general-purpose software, primarily Python. However, real-world software engineering extends well beyond web applications and data science tooling. In particular, specification-driven vertical domains, where code correctness is defined by external normative documents rather than unit tests alone, remain entirely unexamined.

The 5G core network (5GC) is a representative example of such a domain. Defined by hundreds of Technical Specifications (TS) from the 3rd Generation Partnership Project (3GPP), 5GC implementations translate precise protocol semantics into production code decomposed across multiple Network Functions (NFs). Bugs in this domain differ fundamentally from those in typical open-source projects: correctness is

governed by normative protocol text, distributed microservice architectures span multiple NFs communicating via HTTP/2 and the Packet Forwarding Control Protocol (PFCP), and protocol state machines impose complex multi-step procedures for registration, session management, and mobility. These characteristics make 5G software an ideal testbed for probing the capability boundaries of current coding agents.

To this end, we propose SWE-Bench 5G, the first benchmark designed to investigate a concrete question: can AI coding agents resolve real-world bugs in 5G core network software? The benchmark draws task instances from three open-source 5G core implementations, free5GC [2], Open5GS [3], and Magma [4], spanning 7 NFs. Each instance is packaged as a self-contained Docker environment with fail-to-pass tests, and a dual test strategy is introduced to handle the complex runtime dependencies unique to telecom code. In addition, since many original issues cite specific 3GPP specification clauses, we leverage these references to construct concise specification context documents, enabling a controlled evaluation of whether providing domain knowledge improves agent performance.

Our contributions are threefold:

- 1) A benchmark framework mapping real 5G bugs to the SWE-Bench task format, with a dual test strategy for specification-driven code and Docker-based reproducibility.
- 2) An evaluation of four LLMs under single-turn and multi-turn agent paradigms, revealing that iterative code editing capability, not domain knowledge, is the primary bottleneck.
- 3) A controlled experiment on 3GPP specification injection, demonstrating that domain knowledge improves resolve rates on specification-dependent bugs but provides no benefit on generic defensive checks.

II. RELATED WORK

A. From SWE-Bench to Domain-Specific Benchmarks

The evaluation of AI coding agents has progressed through several stages. SWE-Bench [1] pioneered this direction by drawing 2,294 task instances from 12 popular Python repositories on GitHub. Each instance pairs an issue description with a fail-to-pass test, enabling automated binary evaluation of whether an agent can produce a correct patch. Early frontier

Corresponding author: Jianhua Tang (jtang4@e.ntu.edu.sg). J. Chen (202110190459@mail.scut.edu.cn) and J. Tang are with Shenzhen Smart City Technology Development Group Company, Ltd. X. Yang (202510192907@mail.scut.edu.cn) is with the Shien-Ming Wu School of Intelligent Engineering, South China University of Technology. Z. Lv (lvzh67@chinaunicom.cn) is with China Unicom Group Co., Ltd.

¹Dataset: <https://huggingface.co/datasets/tenderzada/SWEBench5G>.

models achieved single-digit resolve rates on this benchmark, catalyzing a wave of agent architecture research.

Subsequent refinements improved evaluation quality. SWE-Bench Lite selected a 300-instance subset for faster iteration, while SWE-Bench Verified introduced human validation to remove ambiguous instances. SWE-Bench Multimodal [?] extended evaluation to visual software domains, requiring agents to process screenshots and UI specifications alongside code. These efforts remained within the Python ecosystem.

B. Expanding Language and Task Scope

A second generation of benchmarks broadened the language and task coverage. SWE-Bench Mobile [5] moved to iOS development with a 500K-line Swift/Objective-C codebase, introducing diff-based intent tests that verify patch correctness through structural source code inspection rather than runtime execution. This addressed the challenge of testing functions with complex runtime dependencies, an approach we adapt for 5G NF functions that require SCTP connections, MongoDB contexts, or inter-NF signaling.

BeyondSWE [6] broadened the resolution scope further to include cross-repository reasoning, domain-specific problem solving, and dependency-driven migrations, packaging each of its 500 instances in a Docker image for full reproducibility. Their experiments revealed that even frontier models plateau below 45% success on these broader tasks, compared to 80%+ on SWE-Bench Verified. We adopt the same containerized design and extend it to the telecommunications domain.

C. Skill Injection and Domain Knowledge

A complementary research direction investigates whether external knowledge can improve agent performance. SWE-Skills-Bench [7] formalized this question by defining 49 programming skills and evaluating their injection into coding agents. Their central finding is that 80% of skills provide zero improvement to resolve rates, with utility depending on domain match, abstraction level, and context compatibility. Skills that are too abstract or too specific both fail to help; only skills that closely match the task domain and provide actionable guidance show positive effects.

This finding motivates our investigation of 3GPP specifications as skill documents. Unlike generic coding guidelines, 3GPP Technical Specifications define the authoritative ground truth for correct 5GC behavior and may therefore represent a category of high-utility, domain-matched skill.

D. Gap in Telecommunications

Despite the growing diversity of benchmarks, all existing work targets general-purpose software. No benchmark evaluates agents on protocol-specified, distributed telecommunications systems. The 5G domain introduces challenges that are absent from Python-centric benchmarks: specification-driven correctness criteria, distributed NF coordination, and domain knowledge underrepresented in LLM training data relative to web development or data science.

TABLE I
SOURCE PROJECTS FOR SWE-BENCH 5G.

Project	Language	Repos	Candidates	Validated
free5GC [2]	Go	20	320	128
Open5GS [3]	C	1	140	58
Magma [4]	Go/Python	12	95	24
Total		33	555	210

III. BENCHMARK DESIGN

A. Overview

Fig. 1 illustrates the SWE-Bench 5G evaluation pipeline. In Phase 1, an AI agent receives an issue description and the NF source code inside a Docker container, optionally augmented with a 3GPP specification excerpt, and produces a patch. In Phase 2, the patch is applied in a fresh container and validated against the test suite.

B. Task Formulation

Each task instance is a tuple $\mathcal{T} = (I, O, E)$ where I is the input comprising a problem statement with optional 3GPP references and the NF source at the buggy commit, O is the expected output in the form of a unified diff patch, and E is the evaluation consisting of `PASS_TO_PASS` and `FAIL_TO_PASS` test suites. An instance is resolved if and only if all fail-to-pass (F2P) tests pass and all pass-to-pass (P2P) tests remain passing after applying the patch.

C. Source Projects

We draw task instances from three open-source 5G core network implementations, summarized in Table I, to improve generalizability and language diversity.

free5GC is a Go implementation of the 3GPP Release 15+ 5GC with a modular architecture of one repository per NF. From free5GC alone, validated instances cover seven NFs: Access and Mobility Management Function (AMF), Policy Control Function (PCF), Session Management Function (SMF), Unified Data Management (UDM), NF Repository Function (NRF), Network Slice Selection Function (NSSF), and Authentication Server Function (AUSF). Open5GS is a C implementation widely deployed in research testbeds, with a monolithic repository containing all NFs. Magma, originally developed by Meta, provides a hybrid Go/Python 5G core oriented toward access gateway functionality. The three projects collectively provide language diversity across Go, C, and Python, as well as architectural diversity ranging from modular to monolithic designs.

D. Dual Test Strategy

Many 5G NF functions cannot be unit-tested directly due to complex runtime dependencies such as Stream Control Transmission Protocol (SCTP) connections, MongoDB sessions, and inter-NF signaling. We therefore employ two complementary test strategies.

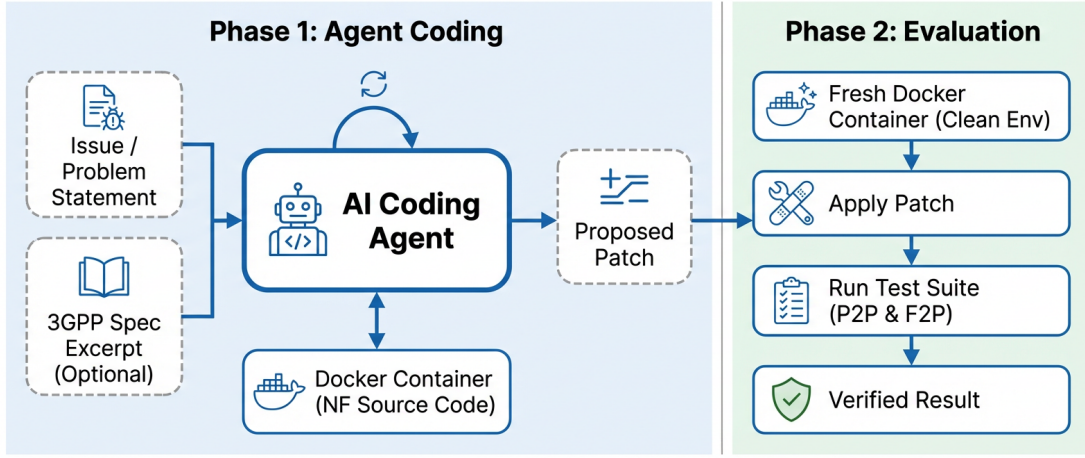


Fig. 1. SWE-Bench 5G evaluation pipeline. In Phase 1 the agent reads the issue and NF source code, optionally augmented with a 3GPP specification excerpt, and produces a patch. In Phase 2 the patch is applied in a fresh container and validated against pass-to-pass (P2P) and fail-to-pass (F2P) tests.

Strategy A, Direct Call, applies when the buggy function accepts simple arguments. We call it directly with crash-triggering inputs and use Go’s `defer/recover` mechanism to detect panics. For example, a PCF policy authorization function can be invoked with a `nil routeReq` parameter, and the test catches the resulting panic:

```
defer func(){ if r:=recover(); r!=nil {
t.Fatalf("BUG: %v",r) } }()
```

Strategy B, Diff-Based Intent, inspired by SWE-Bench Mobile [5], verifies that the source code contains the expected fix pattern rather than testing runtime behavior. For example, a test may read the source file and check whether a `nil` guard such as `GNbId != nil` appears before a pointer dereference. This forces agents to modify the actual source code while avoiding the construction of complex mock objects. The distribution of strategies across instances is detailed in Table II.

E. Validation Criteria

Each task instance must satisfy a three-step validation before inclusion in the benchmark. First, all existing tests must pass at the parent commit, confirming that the Docker environment is correctly configured. Second, the F2P tests must fail at the parent commit, confirming that the bug is present and detectable. Third, after applying the ground-truth fix via the merged pull request, all tests must pass. Only instances that satisfy all three conditions are included. This process ensures that every instance represents a genuine, reproducible bug with a verified fix.

F. Dataset Construction Pipeline

Our automated pipeline operates in five stages: (1) mine closed issues with linked merged pull requests from 33 repositories via the GitHub API, yielding 555 candidates; (2) filter by quality criteria including clear descriptions and source-modifying PRs; (3) identify parent (buggy) and fix commits; (4) construct fail-to-pass tests using Strategy A or B; (5) batch-build Docker images and run three-step validation

TABLE II
SWE-BENCH 5G DATASET STATISTICS.

Attribute	Value
Validated instances	210
Source projects	free5GC, Open5GS, Magma
Languages	Go, C, Python
NFs covered	7
Bug types	nil/null pointer (89), crash (42), missing validation (35), logic error (27), concurrency (17)
Difficulty	Easy 126, Medium 62, Hard 22
Test strategies	68 Direct Call + 142 Diff-Based
Avg. tests/instance	3.4
3GPP specs referenced	14 Technical Specifications
Docker base images	golang:1.25, gcc:14, python:3.12

(existing tests pass, F2P tests fail at parent, all tests pass after fix). Only instances passing all checks are included. Table II summarizes the resulting dataset.

IV. EXPERIMENTAL SETUP

A. Agent Paradigms

We evaluate two agent paradigms. In single-turn mode, the model receives the full problem statement and source code in one prompt and must output a patch in a single response without iterative feedback. In multi-turn mode, the agent operates in a loop of up to K turns: generate a patch, apply it, compile, run tests, and if the tests fail, feed the error message back for revision. We set $K=5$.

B. Models

We evaluate four LLMs selected for diversity across providers, as listed in Table III. All models are accessed through OpenAI-compatible APIs.

C. Specification-as-Skill Protocol

In the context of coding agent evaluation, a skill is a structured reference document that encodes domain-specific

TABLE III
MODELS EVALUATED IN OUR EXPERIMENTS.

Model	Provider	API	Paradigm
Qwen3.5-Flash	Alibaba	DashScope	Single / Multi
Kimi-128k	Moonshot	Moonshot	Multi
Claude Sonnet 4	Anthropic	OpenRouter	Multi
GPT-4.1	OpenAI	OpenRouter	Multi

knowledge, injected into the agent’s context at inference time without any model modification [7]. We adapt this concept to the telecommunications domain by providing 3GPP specification context as skill documents.

Many free5GC and Open5GS issues cite specific 3GPP Technical Specification clauses in the issue description or pull request discussion. For each task instance, we use these references as the starting point to construct a concise specification context document. The document summarizes the relevant protocol semantics, including field optionality constraints and expected NF behavior, and appends a key implication paragraph linking the specification to the task context without revealing the fix. Each resulting document is a self-contained Markdown file averaging 350 tokens. For example, the document for a PCF policy authorization task references TS 29.514, which states that the `AfRoutReq` field is optional when `suppFeat` bit 1 is set.

The evaluation follows a paired A/B design. Condition A provides only the problem statement and source code, while Condition B additionally appends the specification document to the problem statement. Both conditions use identical Docker images, test suites, and evaluation criteria. We evaluate this on a subset of 50 instances covering both generic bugs, where the fix requires only standard defensive programming, and specification-dependent bugs, where the correct behavior is governed by a specific 3GPP clause.

V. RESULTS AND ANALYSIS

A. Main Results

Table IV presents the resolve rates broken down by bug category. All models are evaluated in multi-turn mode ($K=5$) except where noted. We report per-category %Resolved and an overall average. The best result in each column is shown in bold and the second-best is underlined.

Claude Sonnet 4 achieves the highest average resolve rate at 30.0%, followed by GPT-4.1 at 20.0%. Qwen3.5-Flash and Kimi-128k each resolve approximately 10% of instances in multi-turn mode. In single-turn mode ([†]), Qwen3.5-Flash resolves zero instances despite correctly diagnosing bugs in over 90% of cases, illustrating the critical role of iterative feedback. Patch application rates (edits successfully written to source, regardless of test outcome) reach 79.5% for Claude Sonnet 4, 67.1% for GPT-4.1, 57.1% for Qwen3.5-Flash, and 51.4% for Kimi-128k, indicating that the gap between applying a patch and passing tests remains substantial.

Performance varies across bug categories. Nil/null pointer bugs yield the highest resolve rates across all models, as

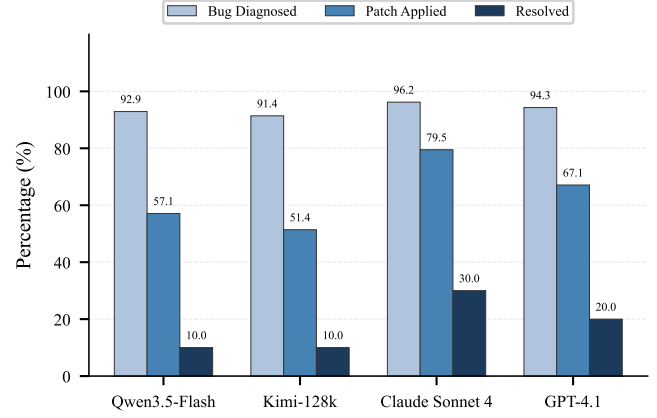


Fig. 2. Multi-turn evaluation results ($K=5$). All models diagnose bugs at rates above 91% but resolve rates range from 10% to 30%, revealing a persistent gap between comprehension and code editing.

these often require inserting a single guard check. Kimi-128k notably outperforms Qwen3.5-Flash on missing validation bugs, achieving 11.4% compared to 8.6%, suggesting that its longer context window helps with specification-dependent fixes. Concurrency bugs are the most challenging category, with even Claude Sonnet 4 resolving only 17.6%.

B. Single-Turn versus Multi-Turn Analysis

Fig. 2 shows the evaluation results across all four models in multi-turn mode. The three-stage funnel, from Bug Diagnosed through Patch Applied to Resolved, is visible for every model. All models diagnose bugs at rates above 91%, but resolve rates remain between 10% and 30%, revealing a persistent gap between bug comprehension and precise code editing.

For Qwen3.5-Flash, the multi-turn loop improves patch application from 34.3% in single-turn mode to 57.1% and enables successful resolution of 10.0% of instances. The feedback mechanism works as intended: when the initial patch causes a compilation error, the error message is fed back, prompting a corrected version in subsequent turns. This result indicates that agentic iteration, not domain knowledge, constitutes the primary bottleneck for telecom software engineering.

The 30% resolve rate of Claude Sonnet 4 on SWE-Bench 5G is notably lower than the rates reported for similar models on general-purpose benchmarks. Contributing factors include the stricter type system of Go and C compared to Python, deeply nested NF function signatures, and the exact-match requirement of our SEARCH/REPLACE patch mechanism.

C. Failure Mode Analysis

We categorize failures into four stages (Table V). Each instance-model pair is classified by the earliest stage at which the agent fails.

The dominant failure mode across all models is incomplete fix, accounting for 72 to 80 instances per model, where the agent addresses part of the bug but misses additional dereference sites or edge cases. Patch format error is the

TABLE IV

MAIN RESULTS ON SWE-BENCH 5G (%RESOLVED). MODELS EVALUATED IN MULTI-TURN MODE ($K=5$) UNLESS NOTED. BEST IN **BOLD**, SECOND-BEST UNDERLINED. [†] SINGLE-TURN (NO FEEDBACK LOOP).

Model	NilPtr (89)	Crash (42)	MissValid (35)	LogicErr (27)	Concur. (17)	AVG %Resolved
Qwen3.5-Flash [†]	0.0	0.0	0.0	0.0	0.0	0.0
Qwen3.5-Flash	12.4	11.9	8.6	3.7	5.9	10.0
Kimi-128k	11.2	9.5	11.4	7.4	5.9	10.0
GPT-4.1	<u>24.7</u>	<u>21.4</u>	<u>14.3</u>	<u>14.8</u>	<u>11.8</u>	<u>20.0</u>
Claude Sonnet 4	38.2	28.6	25.7	18.5	17.6	30.0

TABLE V

FAILURE MODE BREAKDOWN (MULTI-TURN, $K=5$). EACH CELL SHOWS THE NUMBER OF INSTANCES WHOSE EARLIEST FAILURE OCCURS AT THAT STAGE.

Failure Stage	Qwen	Kimi	Claude	GPT
Bug not diagnosed	15	18	8	12
Patch format error	75	84	35	57
Compile failure	20	15	24	22
Incomplete fix	79	72	80	77
Resolved	21	21	63	42

most prominent bottleneck for Qwen and Kimi, at 75 and 84 instances respectively, where the model generates a correct fix idea but produces a SEARCH block that does not exactly match the source whitespace or formatting. Claude and GPT show substantially fewer format errors, at 35 and 57 respectively, suggesting stronger adherence to structured output formats. This distribution indicates that improving patch application robustness through fuzzy matching could disproportionately benefit weaker models without requiring improvements to the underlying LLM.

D. Specification-as-Skill Results

Table VI presents the specification-as-skill results. We evaluate Claude Sonnet 4, the best-performing model, on a subset of 50 instances, grouped by the 3GPP specification used as the skill document. Pass^+ and Pass^- denote resolve rates with and without specification injection, respectively. ΔP is the skill utility delta, C^+ is the average token cost with specification, and ρ is the token overhead ratio.

Three skill domains show positive utility deltas: policy authorization, policy control, and session management. These deltas range from +16.7% to +25.0% and correspond to bugs where the correct behavior is defined by field optionality or value range constraints in the specification. The remaining seven specifications show zero delta, covering bugs that require only generic defensive programming such as nil guards.

The average token overhead is 12%, reflecting the concise nature of curated 3GPP excerpts compared to full specification documents.

These results demonstrate that specification utility is conditional on bug type in the telecommunications domain. 3GPP specifications define the ground truth for correct protocol behavior, making them effective for protocol-semantic bugs,

TABLE VI

SPECIFICATION-AS-SKILL EVALUATION (CLAUDE SONNET 4, MULTI-TURN). $\text{PASS}^+/\text{PASS}^-$: RESOLVE RATE WITH/WITHOUT SPEC. ΔP : UTILITY DELTA. ρ : TOKEN OVERHEAD. BEST VIEWED IN COLOR.

Skill Domain	#Tasks	Pass ⁺	Pass ⁻	ΔP	C^+	ρ
Generic bug type (nil/crash)						
NAS ^a protocol	8	37.5	37.5	0.0	3.4K	+11%
RAN ^b signaling	5	40.0	40.0	0.0	3.2K	+9%
NF discovery	4	25.0	25.0	0.0	3.5K	+13%
Authentication	4	25.0	25.0	0.0	3.1K	+8%
Slice selection	4	25.0	25.0	0.0	3.3K	+10%
Mobility mgmt.	5	40.0	40.0	0.0	3.6K	+14%
Spec-dependent bug type (validation/logic)						
Policy authorization	6	33.3	16.7	+16.7	3.8K	+15%
Policy control	4	25.0	0.0	+25.0	3.7K	+14%
Session mgmt.	5	20.0	0.0	+20.0	3.9K	+16%
System architecture	5	20.0	20.0	0.0	4.1K	+18%
Overall	50	30.0	24.0	+6.0	3.5K	+12%

^aNon-Access Stratum. ^bRadio Access Network.

but they provide no value for bugs requiring only generic defensive programming.

VI. CONCLUSION

We have introduced SWE-Bench 5G, the first benchmark for evaluating AI coding agents on telecom network engineering tasks. The benchmark draws from three open-source 5G core projects, provides a dual test strategy for specification-driven code, Docker-based reproducibility, and an automated construction pipeline. Our evaluation across four LLMs reveals that agentic iteration is the primary bottleneck, with multi-turn feedback improving resolve rates from 0% to 10 to 30%. The specification-as-skill experiment demonstrates that 3GPP protocol excerpts provide targeted improvement on specification-dependent bugs while offering no benefit for generic defensive checks.

Future work includes expanding the dataset with cross-NF coordination tasks, integrating full agentic tools with native file editing capabilities, and evaluating generalization to additional 5G implementations.

REFERENCES

- [1] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, "SWE-bench: Can language models resolve real-world GitHub issues?" in *Proc. ICLR*, 2024.

- [2] free5GC Team, “free5GC: Open source 5G core network,” <https://github.com/free5gc/free5gc>, 2024.
- [3] Open5GS Team, “Open5GS: Open source implementation of 5G core and EPC,” <https://github.com/open5gs/open5gs>, 2024.
- [4] Linux Foundation, “Magma: Platform for building access networks and modular network services,” <https://github.com/magma/magma>, 2024.
- [5] Z. Yao *et al.*, “SWE-bench mobile: An evaluation benchmark for mobile app engineering,” *arXiv preprint arXiv:2602.09540*, 2025.
- [6] Z. Shi *et al.*, “BeyondSWE: A comprehensive benchmark for evaluating code agents beyond narrow bug fixing,” *arXiv preprint arXiv:2603.03194*, 2025.
- [7] T. Han, Y. Zhang, W. Song, C. Fang, Z. Chen, Y. Sun, and L. Hu, “SWE-skills-bench: Do agent skills actually help in real-world software engineering?” *arXiv preprint arXiv:2603.15401*, 2025.